

cse 242 hw2

Ian Chi

April 20, 2026

1

1. (a)

$$\begin{aligned}
 J(w) &= \left| Xw - y \right|_2^2 + \lambda \left| w \right|_2^2 \\
 J(w) &= (Xw - y)^T (Xw - y) + \lambda w^T w \\
 J(w) &= w^T X^T X w - w^T X^T y - y^T X w + y^T y + \lambda w^T w \quad (1) \\
 J(w) &= w^T X^T X w - 2y^T X w + \lambda w^T w \\
 \frac{\partial J(w)}{\partial w} &= 2X^T X w - 2y^T X + 2\lambda w = 0
 \end{aligned}$$

then we see

$$\begin{aligned}
 2X^T X w + 2\lambda w &= 2y^T X \\
 (X^T X + \lambda I)w &= y^T X \quad (2)
 \end{aligned}$$

then we get the solution in the hw.

(b) as $\lambda \rightarrow 0$, $w_{ridge} \rightarrow (X^T X)^{-1} X^T y$. as $\lambda \rightarrow \infty$, $w_{ridge} \rightarrow 0$.
the former is when penalty goes to zero and the latter is when error dominates.

(c)

$$\begin{aligned}
 \left| \hat{X}w - \hat{y} \right| &= (\hat{X}w - \hat{y})^T (\hat{X}w - \hat{y}) \\
 &= w^T \hat{X}^T \hat{X} w - \hat{y}^T \hat{X} w - w^T \hat{X}^T \hat{y} + \left| \hat{y} \right|^2 \\
 &= w^T (X^T X + \sqrt{\lambda} X^T I_d + \sqrt{\lambda} I_d^T X + \lambda I) w \\
 &\quad - \hat{y}^T \hat{X} w - w^T \hat{X}^T \hat{y} + \left| \hat{y} \right|^2 \quad (3)
 \end{aligned}$$

note $(\sqrt{\lambda} I w)^2 = \lambda \sum w_i^2$. factoring out w^T , excluding the $w = 0$ solution, the argmin when zero gives the original expression.

2

(a)

$$\begin{aligned}
 (X^T X + \lambda I)^{-1} &= (Q \Lambda^2 Q^T + \lambda I)^{-1} \\
 &= (Q \Lambda Q^T + \lambda Q I Q^T)^{-1} \\
 &= (Q (\Lambda^2 + \lambda I) Q^T)^{-1} \\
 &= (Q^T)^{-1} ((\Lambda^2 + \lambda I))^{-1} Q^{-1} \\
 &= Q ((\Lambda^2 + \lambda I))^{-1} Q^T
 \end{aligned} \tag{4}$$

λ inversely scales the eigenvalue

(b)

$$\kappa(X^T X) = \frac{\sigma_{max}^2}{\sigma_{min}^2} \tag{5}$$

since sigmas are positive, the inequality on the right holds.

the equality on the left is true because of part a. the eigenvalues are exactly $\Lambda + \lambda I$

we can also see why equality only holds when λ is zero, since every eigenvalue is different on the LHS when λ is nonzero.

(c) if there are zero eigenvalues, it means the matrix is non invertable. the smallest eigenvalues with regularization are λ and no eigenvalues are zero, which implies full rank, which implies invertability.

2. (a) for ols

$$(X^T X)^{-1} X^T y = X^T y \tag{6}$$

for ridge

$$(X^T X + \lambda I)^{-1} X^T y = ((1 + \lambda)I)^{-1} X^T y = \frac{1}{1 + \lambda} X^T y \tag{7}$$

(b) using the hat symbol to denote estimate

$$\begin{aligned}
 E[\hat{w}_{ridge}] &= E \left[\frac{1}{1 + \lambda} X^T y \right] \\
 &= \frac{1}{1 + \lambda} X^T E[y]
 \end{aligned} \tag{8}$$

$$E[y] = E[Xw^* + \epsilon] = Xw^* + E[\epsilon] = Xw^*$$

then

$$E[\hat{w}_{ridge}] = \frac{w^*}{1 + \lambda}$$

similarly

$$\begin{aligned}
Var(\hat{w}_{ridge}) &= Var\left(\frac{1}{1+\lambda}X^Ty\right) \\
&= \left(\frac{1}{1+\lambda}\right)^2 X^T Var(y)(X^T)^T \\
&= \frac{1}{(1+\lambda)^2} X^T Var(y) X \\
&= \frac{\sigma^2}{(1+\lambda)^2} X^T X
\end{aligned} \tag{9}$$

(c)

$$\begin{aligned}
Bias_j &= E[w_{ridge,j}] - w_j^* \\
&= \frac{w_j^*}{1+\lambda} - w_j^* \\
&= w_j^* \left(\frac{1}{1+\lambda} - 1\right) \\
&= w_j^* \left(\frac{1 - (1+\lambda)}{1+\lambda}\right) \\
&= \frac{-\lambda w_j^*}{1+\lambda}
\end{aligned} \tag{10}$$

therefore

$$MSE_j = \left(\frac{\lambda w_j^*}{1+\lambda}\right)^2 + \frac{\sigma^2}{(1+\lambda)^2}$$

to find optimal λ , w wolfram alpha

$$\frac{d}{d\lambda} MSE_j = \frac{2(\lambda(w_j^*)^2 - \sigma^2)}{(1+\lambda)^3}$$

set λ to $\frac{\sigma^2}{(w_j^*)^2}$ to make the numerator 0 and thus the whole term 0.

OLS is bad when noise is high and good when noise is low. ridge is better when noise is high.

3. (a)

$$P(w|X, y) = \frac{P(y|X, w)P(w)}{P(y|X)}$$

$$\ln P(w|X, y) = \ln P(y|X, w) + \ln P(w) - \ln P(y|X)$$

both terms are normal distributions so we see

$$\ln P(y|X, w) = -\frac{1}{2\sigma^2} \|y - Xw\|_2^2 + C_1$$

$$\ln P(w) = -\frac{1}{2\tau^2} \|w\|_2^2 + C_2$$

note, after taking log the constant normalization terms for P become additive constants.

thus

$$\ln P(w|X, y) = -\frac{1}{2\sigma^2} \|y - Xw\|_2^2 - \frac{1}{2\tau^2} \|w\|_2^2 + C$$

but this is just the negative of the ridge objective up to constants so maximizing this is the same as minimizing the ridge constant obj.

- (b) if λ is large, τ^2 is small since $\lambda = \left(\frac{\sigma}{\tau}\right)^2$. the gaussians have small variance and are near zero. since λ controls the weight given to the prior by influencing τ , it exhibits similar behavior.

3

1. (a)

$$\begin{aligned}
 J(w) &= (Xw - y)^T(Xw - y) + \lambda w^T w \\
 &= (w^T X^T X w - w^T X^T y - y^T X w + y^T y) + \lambda w^T w \quad (11) \\
 &= (w^T X^T X w - 2w^T X^T y + y^T y) + \lambda w^T w
 \end{aligned}$$

$$\nabla_w J(w) = 2X^T X w - 2X^T y + 2\lambda w \quad (12)$$

- (b) see jupyter notebook
 - (c) 0.001 converges the fastest. 0.01 diverges. 0.001 goes fast enough but does not overshoot. 0.01 overshoots too much.
 - (d) see jupyter notebook
they are not that close.
2. (a) if a step in the gradient descent hits exactly $w_i = 0$, the gradient of that point will be undefined which breaks the algorithm.
 - (b) see jupyter notebook
 - (c) most weights are zero. the higher the lambda, the lower the fraction of non-zero weights
 - (d) lasso creates far sparser solutions. lasso does a better job of recovering nonzero entries. the force that lasso exerts on weights is constant no matter how negative or positive they are from their optimal while ridge slows down the force on weights as they near zero making it take a very long time for them to reach zero.

4

1. (a) lasso can be used in medicine identifying features that cause disease
 - (b) finance can see factors that cause changes in asset pricing
 - (c) in natural language processing something like this can be used to find words that are important
 - (d) ridge can be used by economists to find the relations between macroeconomic factors, keeping dependence on different factors without losing them completely like lasso might
2. i used gemini for help w plotting
 3. gemini fast, used a different chrome tab, no copilot
 4. it took about 10-12 hours. most of the time was spent coding and verifying answers since they seemed counterintuitive
 5. i think the way that lasso behaves seems odd since the loss doesn't go to zero. also the weight differences for ridge don't go to zero either which i'm not sure if that's a mistake or expected.

P2.Q1

In [463...

```

import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# dimensions
n = 50
d = 100
k = 10

X = np.random.normal(0, 1, size=(n, d))

w_true = np.zeros(d)
non_zero_indices = np.random.choice(d, k, replace=False)
w_true[non_zero_indices] = np.random.normal(0, np.sqrt(5), size=k)

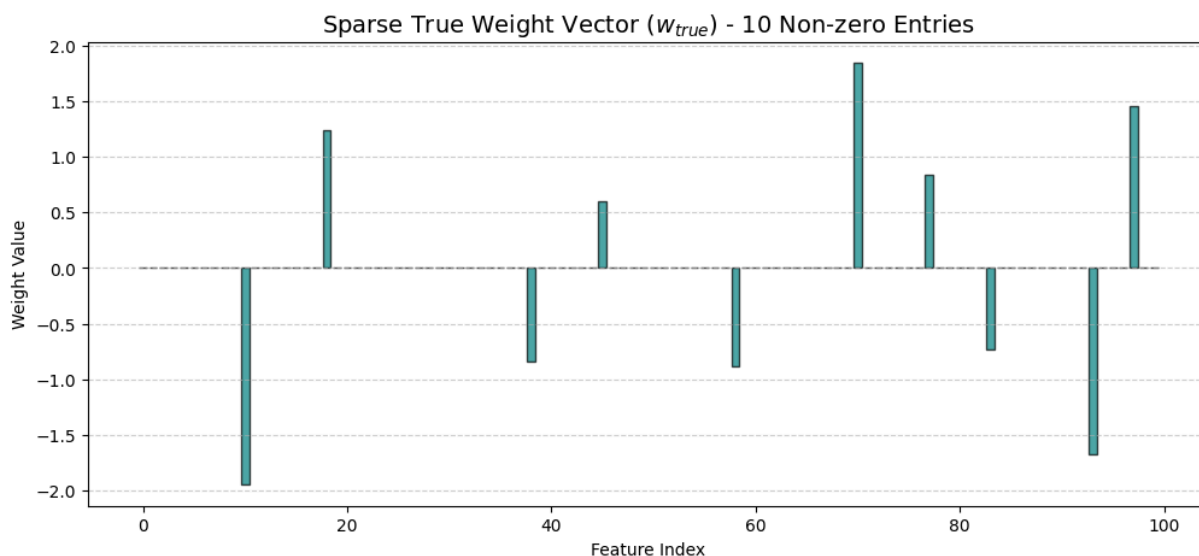
# generate noise epsilon

eps_std = .5
epsilon = np.random.normal(0, eps_std, size=n)

# generate y
y = X @ w_true + epsilon

# Plotting
plt.figure(figsize=(12, 5))
plt.bar(range(d), w_true, color='teal', edgecolor='black', alpha=0.7)
plt.title(f"Sparse True Weight Vector ( $w_{true}$ ) - {k} Non-zero Entries", fontstyle='italic')
plt.xlabel("Feature Index")
plt.ylabel("Weight Value")
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.show()

```



P2.Q2

In [464...

```

import numpy as np
import matplotlib.pyplot as plt

# center
X_mean = np.mean(X, axis=0)
y_mean = np.mean(y)
X_centered = X - X_mean
y_centered = y - y_mean

# closed form
# w_ridge = (X^T X + λI)^-1 X^T y

lambda = 1.0
I = np.eye(d)
w_ridge = np.linalg.inv(X_centered.T @ X_centered + lambda * I) @ X_centered.T @ y_c

y_pred_centered = X_centered @ w_ridge

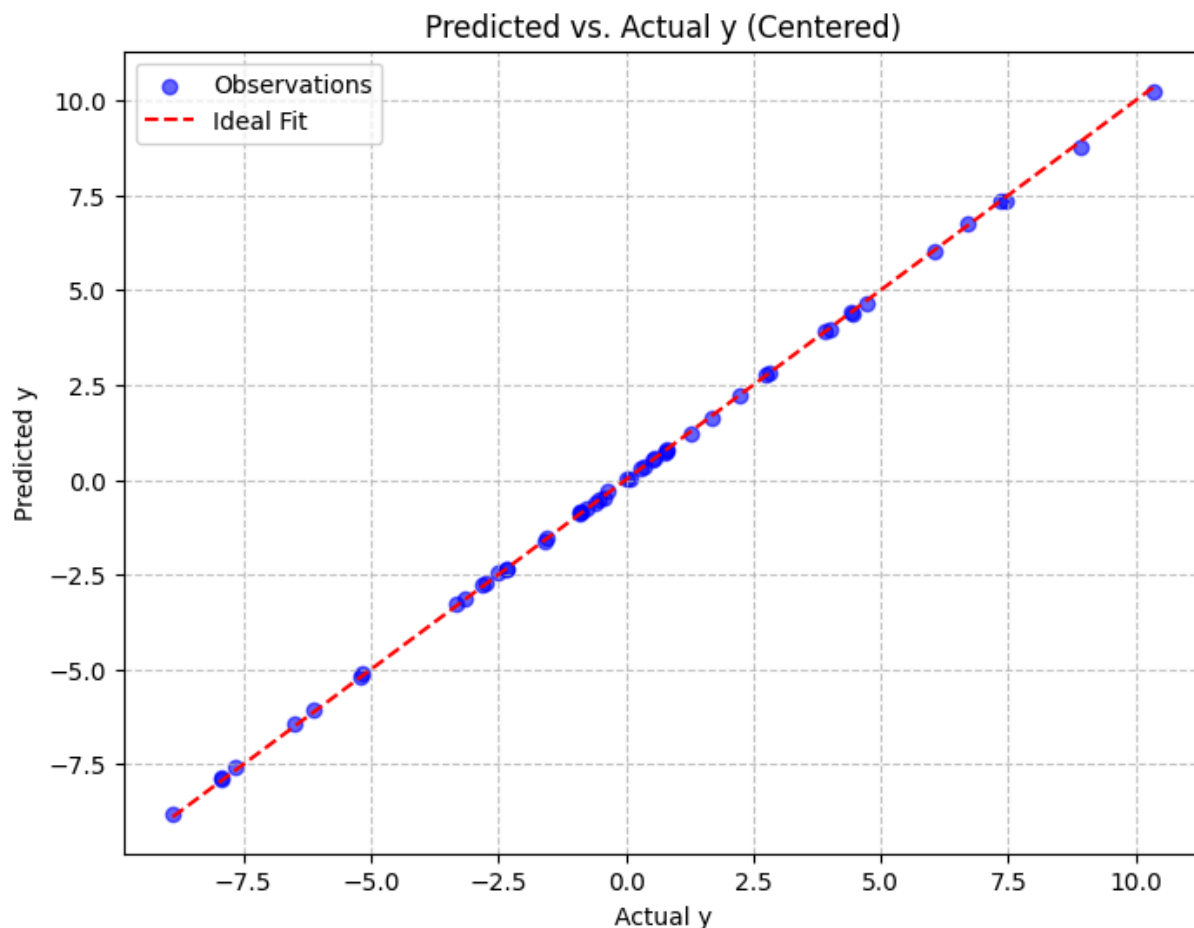
```

In [465...

```

plt.figure(figsize=(8, 6))
plt.scatter(y_centered, y_pred_centered, color='blue', alpha=0.6, label='Observatio
plt.plot([y_centered.min(), y_centered.max()], [y_centered.min(), y_centered.max()])
plt.title('Predicted vs. Actual y (Centered)')
plt.xlabel('Actual y')
plt.ylabel('Predicted y')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)

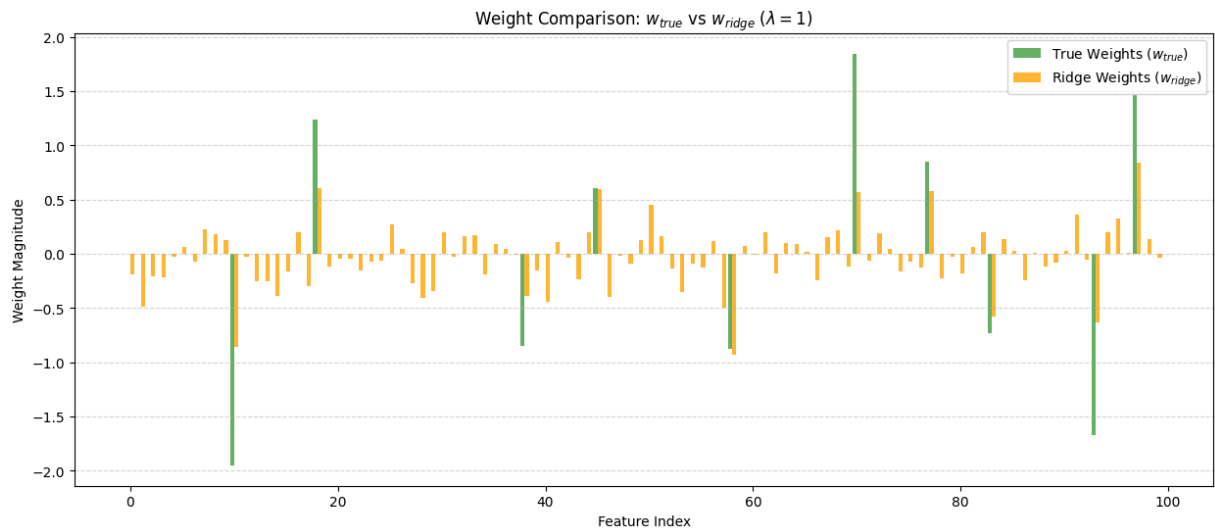
```



In [466...

```
plt.figure(figsize=(15, 6))
indices = np.arange(100)
width = 0.4
plt.bar(indices - width/2, w_true, width, label='True Weights ($w_{true}$)', color=
plt.bar(indices + width/2, w_ridge, width, label='Ridge Weights ($w_{ridge}$)', col
plt.title('Weight Comparison: $w_{true}$ vs $w_{ridge}$ ($\lambda=1$)')
plt.xlabel('Feature Index')
plt.ylabel('Weight Magnitude')
plt.legend()
plt.grid(axis='y', linestyle='--', alpha=0.5)
```

```
<>:6: SyntaxWarning: invalid escape sequence '\l'
<>:6: SyntaxWarning: invalid escape sequence '\l'
C:\Users\Ian\AppData\Local\Temp\ipykernel_29596\2156376713.py:6: SyntaxWarning: inva
lid escape sequence '\l'
plt.title('Weight Comparison: $w_{true}$ vs $w_{ridge}$ ($\lambda=1$)')
```

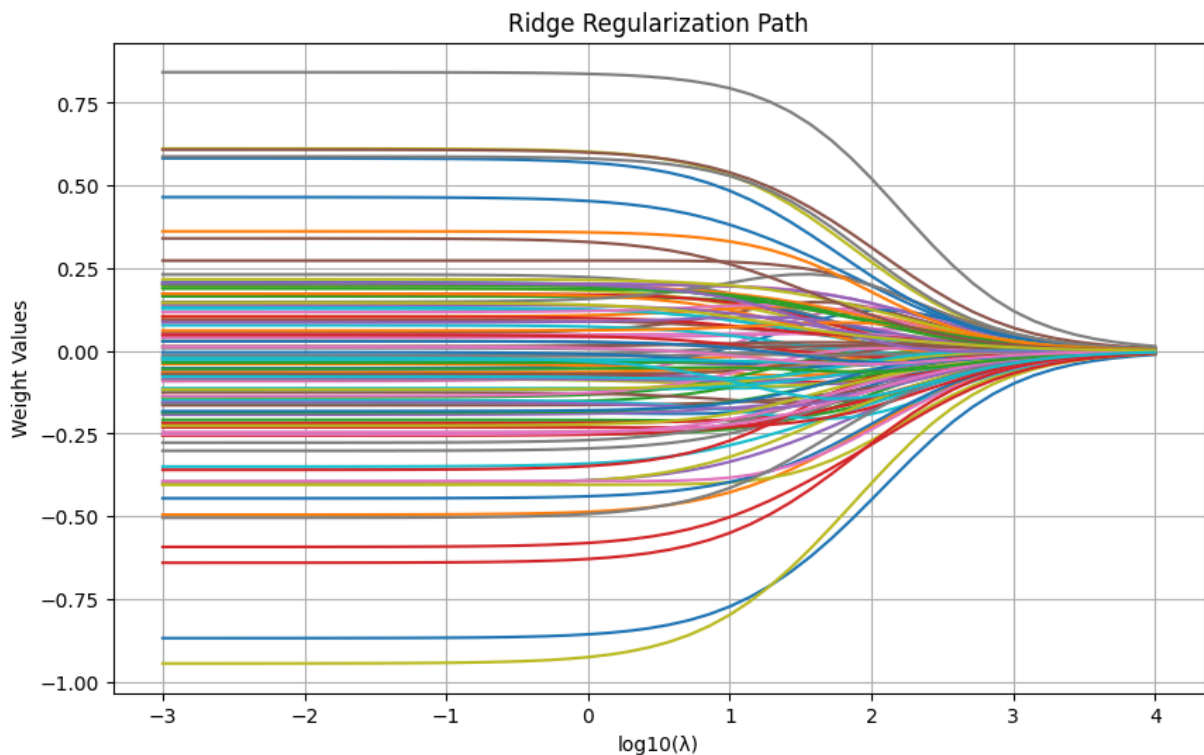
there exists a good correlation between pred and actual. regularization helps prevent overfitting. it does shrink nonzero weights. there are also no exact zeros.

P2.Q3a

```
In [467... lambdas = np.logspace(-3, 4, 50)
weights_path = []
I = np.eye(d)
for lmbda in lambdas:
    w_ridge = np.linalg.inv(X_centered.T @ X_centered + lmbda * I) @ X_centered.T @
    weights_path.append(w_ridge)

weights_path = np.array(weights_path)

plt.figure(figsize=(10, 6))
plt.plot(np.log10(lambdas), weights_path)
plt.title('Ridge Regularization Path')
plt.xlabel('log10(λ)')
plt.ylabel('Weight Values')
plt.grid(True)
plt.savefig('ridge_regularization_path.png')
```



as λ grows, the weights converge.

In [468...

```
indices = np.arange(n)
np.random.shuffle(indices)
train_idx = indices[:int(0.8 * n)]
test_idx = indices[int(0.8 * n):]

X_train, X_test = X[train_idx], X[test_idx]
y_train, y_test = y[train_idx], y[test_idx]

train_mse = []
test_mse = []

for lambda in lambdas:
    #center
    X_train_mean = np.mean(X_train, axis=0)
    y_train_mean = np.mean(y_train)
    X_train_c = X_train - X_train_mean
    y_train_c = y_train - y_train_mean
    X_test_c = X_test - X_train_mean
    y_test_c = y_test - y_train_mean

    XTX = X_train_c.T @ X_train_c
    XTy = X_train_c.T @ y_train_c
    w_ridge = np.linalg.inv(XTX + lambda * I) @ XTy

    y_train_pred = X_train_c @ w_ridge
    y_test_pred = X_test_c @ w_ridge

    # MSE Calculations
    mse_tr = np.mean((y_train_c - y_train_pred)**2)
```

```

mse_te = np.mean((y_test_c - y_test_pred)**2)

train_mse.append(mse_tr)
test_mse.append(mse_te)

# Plotting MSE vs Log10(lambda)
plt.figure(figsize=(10, 6))
plt.plot(np.log10(lambdas), train_mse, label='Training MSE', marker='o', markersize=10)
plt.plot(np.log10(lambdas), test_mse, label='Test MSE', marker='s', markersize=4)
plt.title('Ridge Regression: Training and Test MSE vs. Regularization Strength', fo
plt.xlabel('$\log_{10}(\lambda)$', fontsize=12)
plt.ylabel('Mean Squared Error (MSE)', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.savefig('mse_vs_lambda.png')

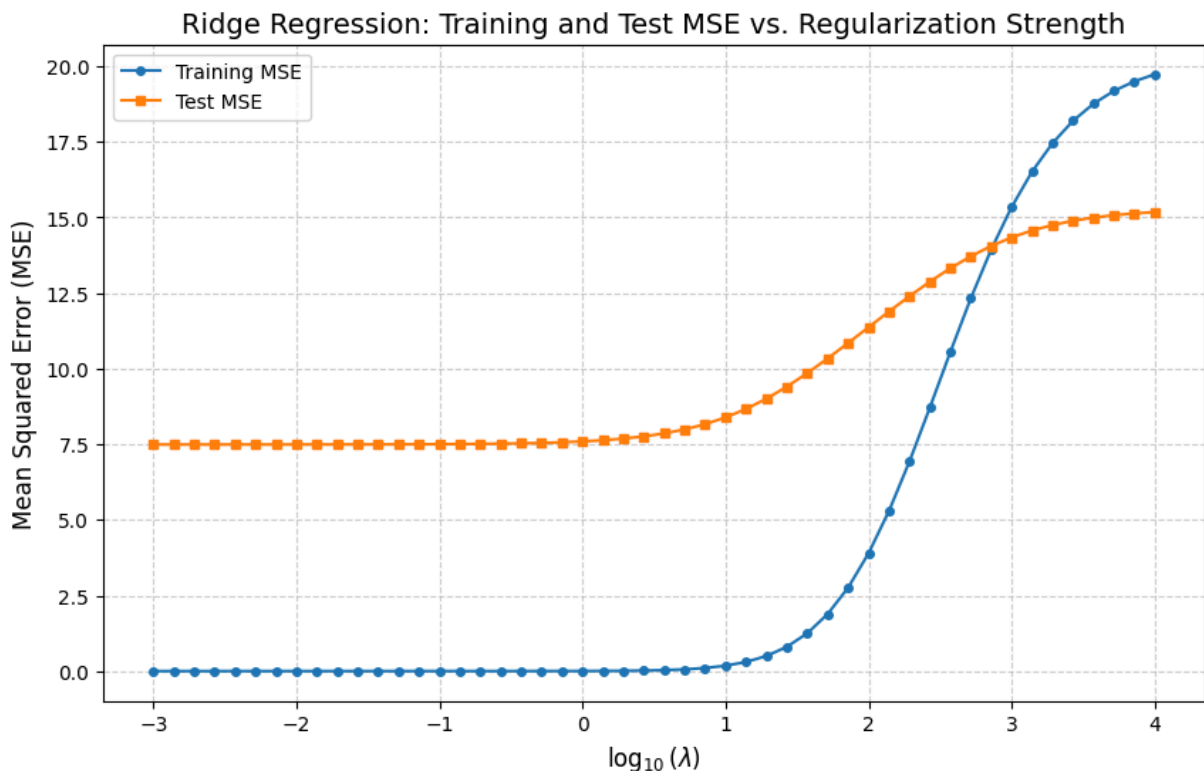
# Find optimal lambda for context
opt_lambda = lambdas[np.argmin(test_mse)]
print(f"Optimal lambda (min test MSE): {opt_lambda:.4f}")
print(f"log10(Optimal lambda): {np.log10(opt_lambda):.4f}")

```

```

<>:41: SyntaxWarning: invalid escape sequence '\l'
<>:41: SyntaxWarning: invalid escape sequence '\l'
C:\Users\Ian\AppData\Local\Temp\ipykernel_29596\3144364458.py:41: SyntaxWarning: inv
alid escape sequence '\l'
plt.xlabel('$\log_{10}(\lambda)$', fontsize=12)
Optimal lambda (min test MSE): 0.0010
log10(Optimal lambda): -3.0000

```



yes, mse does increase monotonically w λ .

i do not see the behavior of mse decreasing then increasing.

if i up epsilon, i do see the dip.

the optimal is closer to 10^4 after i increase epsilon to 10.

P3.Q1b

In [469...

```
def ridge_gradient_descent(X, y, alpha, lambda, iterations=1000):
    """
    X: Feature matrix (n_samples, n_features)
    y: Target vector (n_samples, 1)
    alpha: Learning rate
    lambda: Regularization parameter
    """
    n_samples, n_features = X.shape
    w = np.zeros((n_features, 1))
    loss_history = []

    for i in range(iterations):
        predictions = X @ w

        #  $J(w) = (Xw - y)^T (Xw - y) + \lambda w^T w$ 
        error = predictions - y
        loss = (error.T @ error) + lambda * (w.T @ w)
        loss_history.append(loss.flatten()[0])

        #  $grad = 2X^T(Xw - y) + 2\lambda w$ 
        gradient = 2 * (X.T @ error) + 2 * lambda * w

        w = w - alpha * gradient

    return w, loss_history

def plot3(losses, iters, alpha, lambda=""):
    plt.plot(range(iters), losses, color='blue', linewidth=2)
    plt.xlabel('Iterations')
    plt.ylabel('Loss  $J(w)$ ')
    plt.title(f'Ridge Regression Gradient Descent Convergence, alpha = {alpha}, lam')
    plt.grid(True, linestyle='--', alpha=0.7)
    # plt.savefig('ridge_loss_plot.png')

def weight_comparison(weights):
    plt.figure(figsize=(15, 6))
    indices = np.arange(100)
    width = 0.4
    plt.bar(indices - width/2, w_true, width, label='True Weights ( $w_{true}$ )', co
    plt.bar(indices + width/2, weights, width, label='Grad Descent Weights ( $w_{rid$ 
    plt.title('Weight Comparison:  $w_{true}$  vs weights')
    plt.xlabel('Feature Index')
    plt.ylabel('Weight Magnitude')
    plt.legend()
    plt.grid(axis='y', linestyle='--', alpha=0.5)
```

In [470...

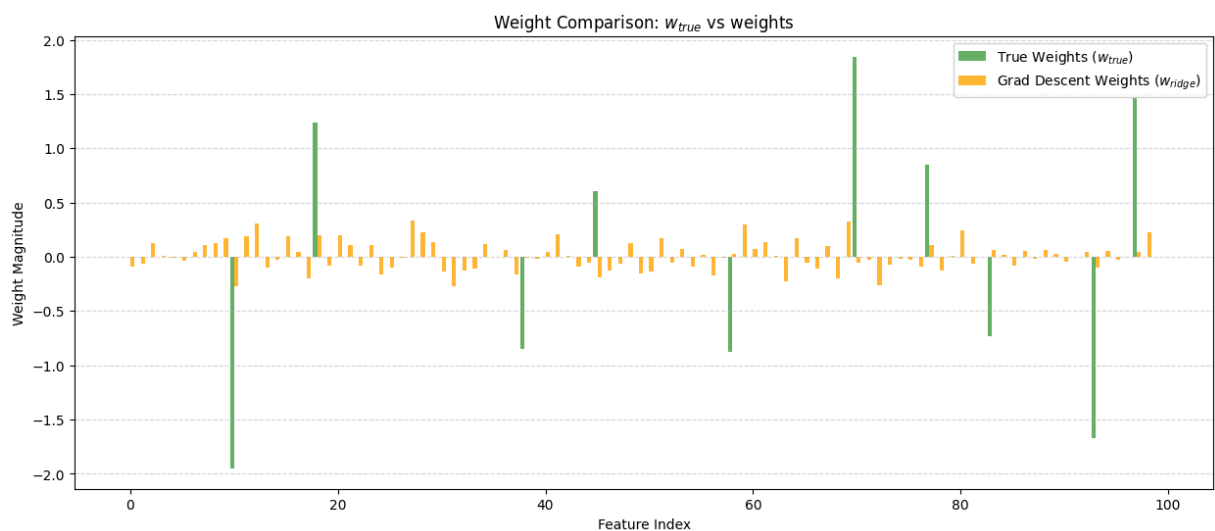
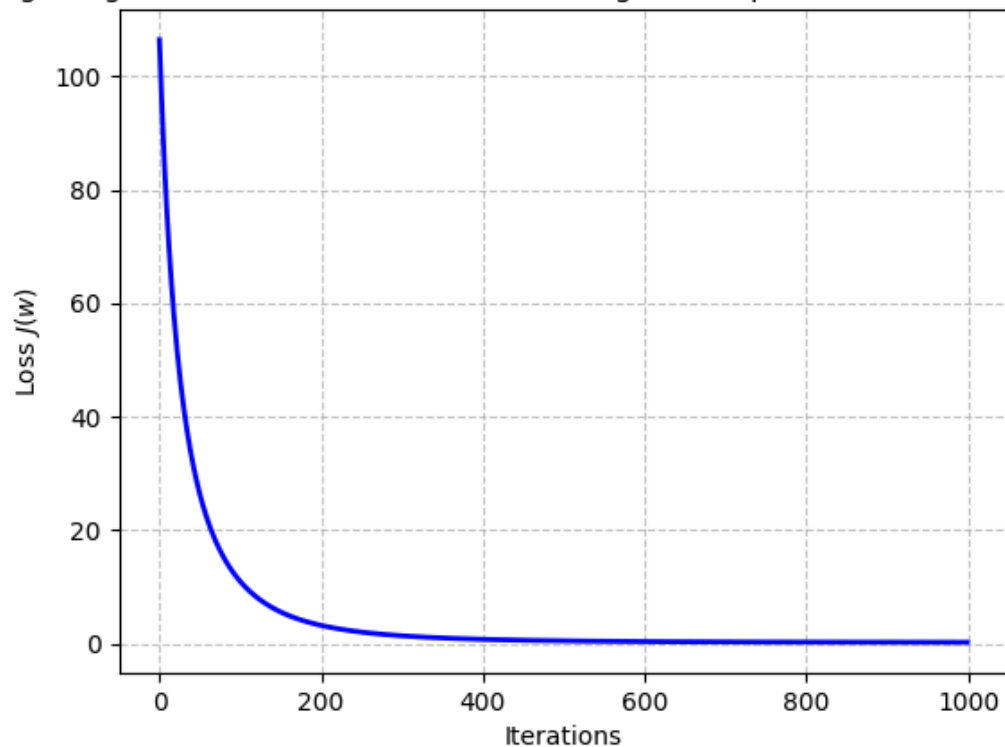
```
alpha1 = [0.0001, 0.001, 0.01]
```

```
In [471... for i in range(len(alpha1)):
    alpha_post_grad_descent_weights, alpha_post_grad_descent_losses = ridge_gradient
    final_weights = alpha_post_grad_descent_weights[:, 0]
    print(f"weight diff: {np.linalg.norm(final_weights - w_true)}")
    plot3(alpha_post_grad_descent_losses, len(alpha_post_grad_descent_losses), alpha
    plt.show()

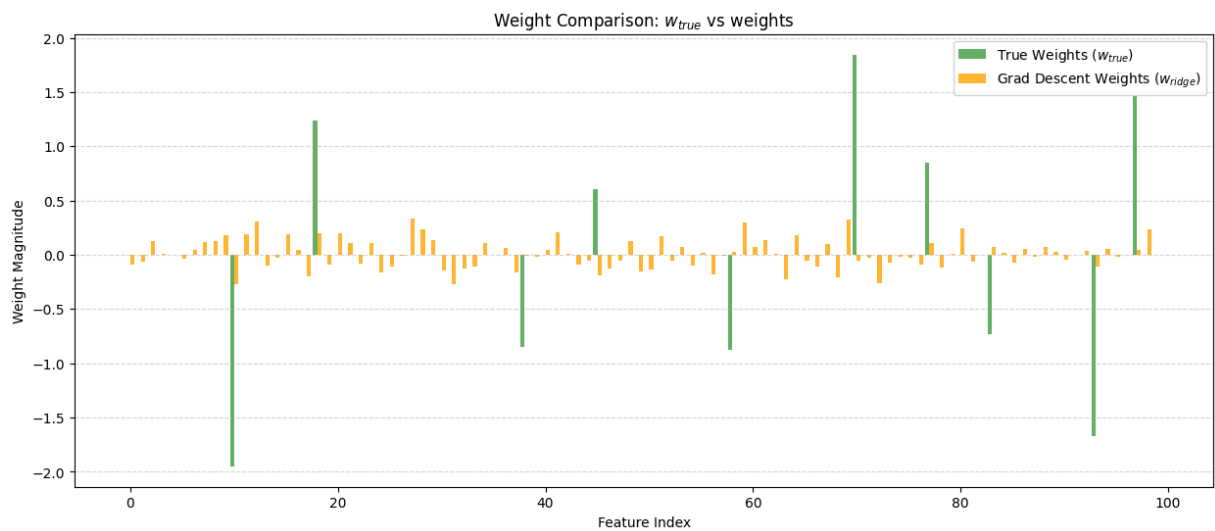
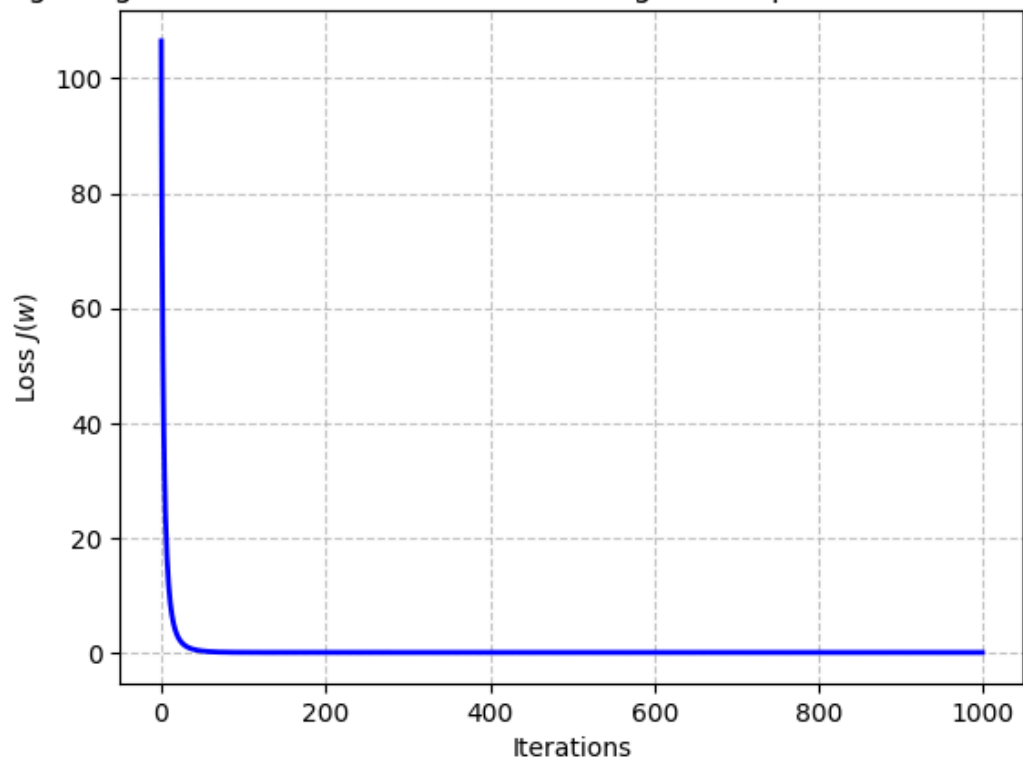
    weight_comparison(final_weights)
    plt.show()
```

weight diff: 4.106559339830438

Ridge Regression Gradient Descent Convergence, $\alpha = 0.0001$, $\lambda = 0.1$



weight diff: 4.108756394526143

Ridge Regression Gradient Descent Convergence, $\alpha = 0.001$, $\lambda = 0.1$ 

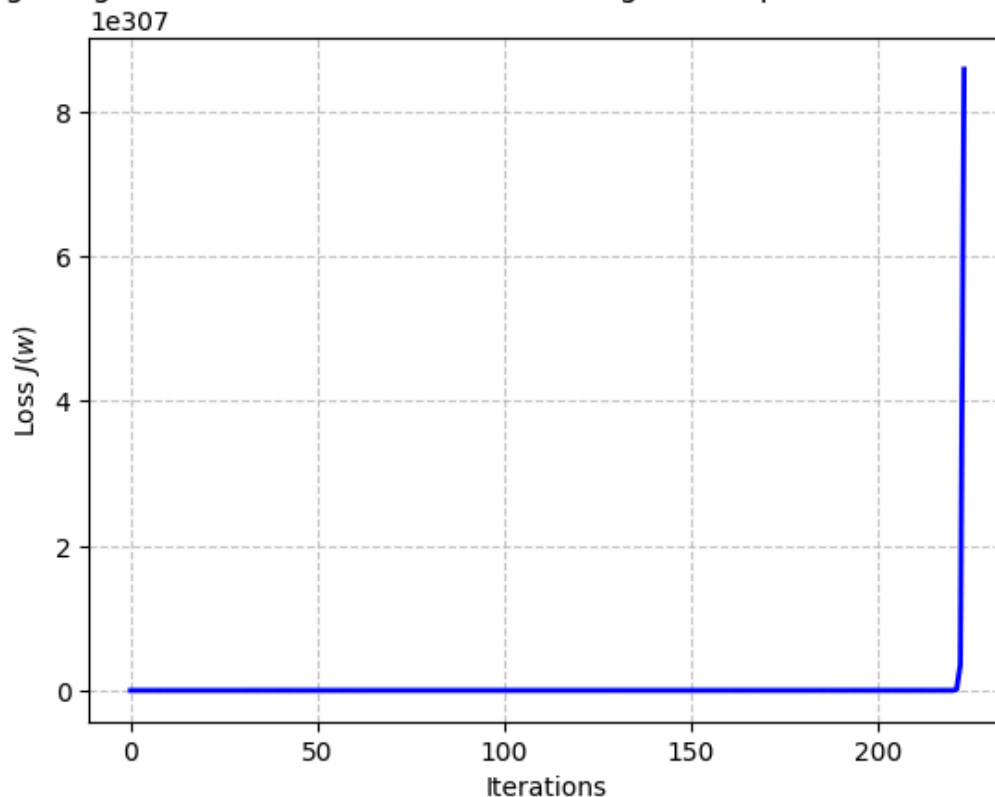
weight diff: nan

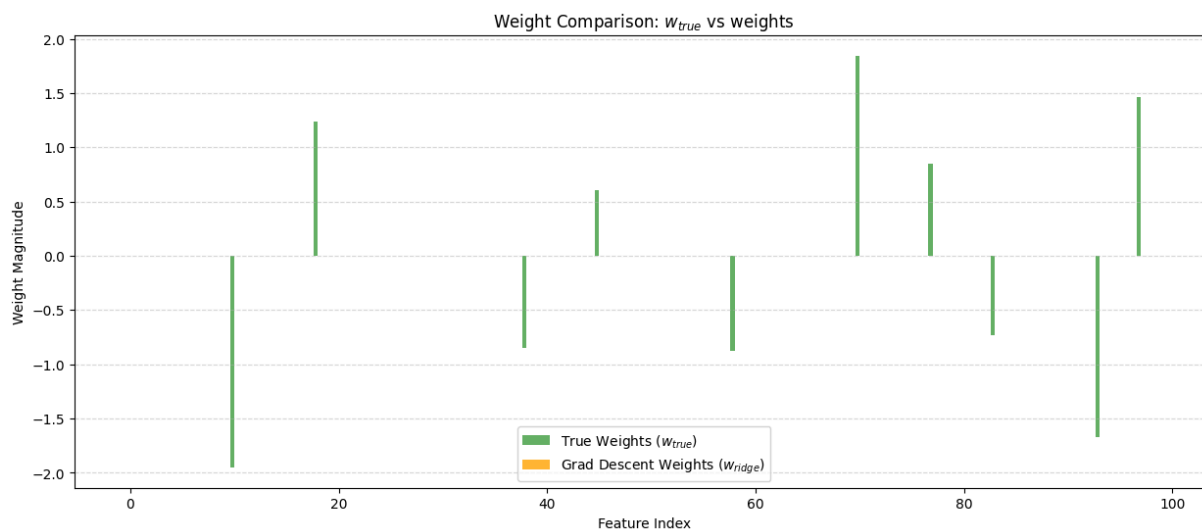
```

C:\Users\Ian\AppData\Local\Temp\ipykernel_29596\111275478.py:17: RuntimeWarning: overflow encountered in matmul
    loss = (error.T @ error) + lambda * (w.T @ w)
C:\Users\Ian\AppData\Local\Temp\ipykernel_29596\111275478.py:17: RuntimeWarning: overflow encountered in add
    loss = (error.T @ error) + lambda * (w.T @ w)
C:\Users\Ian\AppData\Local\Temp\ipykernel_29596\111275478.py:21: RuntimeWarning: overflow encountered in multiply
    gradient = 2 * (X.T @ error) + 2 * lambda * w
C:\Users\Ian\AppData\Local\Temp\ipykernel_29596\111275478.py:13: RuntimeWarning: invalid value encountered in matmul
    predictions = X @ w
C:\Users\Ian\AppData\Local\Temp\ipykernel_29596\111275478.py:17: RuntimeWarning: invalid value encountered in matmul
    loss = (error.T @ error) + lambda * (w.T @ w)
C:\Users\Ian\AppData\Local\Temp\ipykernel_29596\111275478.py:21: RuntimeWarning: overflow encountered in matmul
    gradient = 2 * (X.T @ error) + 2 * lambda * w
C:\Users\Ian\AppData\Local\Temp\ipykernel_29596\111275478.py:21: RuntimeWarning: invalid value encountered in matmul
    gradient = 2 * (X.T @ error) + 2 * lambda * w
C:\Users\Ian\AppData\Local\Temp\ipykernel_29596\111275478.py:21: RuntimeWarning: overflow encountered in add
    gradient = 2 * (X.T @ error) + 2 * lambda * w
C:\Users\Ian\AppData\Local\Temp\ipykernel_29596\111275478.py:23: RuntimeWarning: invalid value encountered in subtract
    w = w - alpha * gradient

```

Ridge Regression Gradient Descent Convergence, $\alpha = 0.01$, $\lambda=0.1$





P3.Q2b

In [472...

```
def lasso_gradient_descent(X, y, alpha, lambda, iterations=1000):
    """
    X: Feature matrix (n_samples, n_features)
    y: Target vector (n_samples, 1)
    alpha: Learning rate
    lambda: Regularization parameter (L1 penalty)
    """
    n_samples, n_features = X.shape
    w = np.zeros((n_features, 1))
    loss_history = []

    for i in range(iterations):
        predictions = X @ w
        error = predictions - y

        l1_penalty = lambda * np.sum(np.abs(w))
        loss = (error.T @ error) + l1_penalty
        loss_history.append(loss.flatten()[0])

        subgradient = 2 * (X.T @ error) + lambda * np.sign(w)

        # Update weights
        w = w - alpha * subgradient

    return w, loss_history
```

In [473...

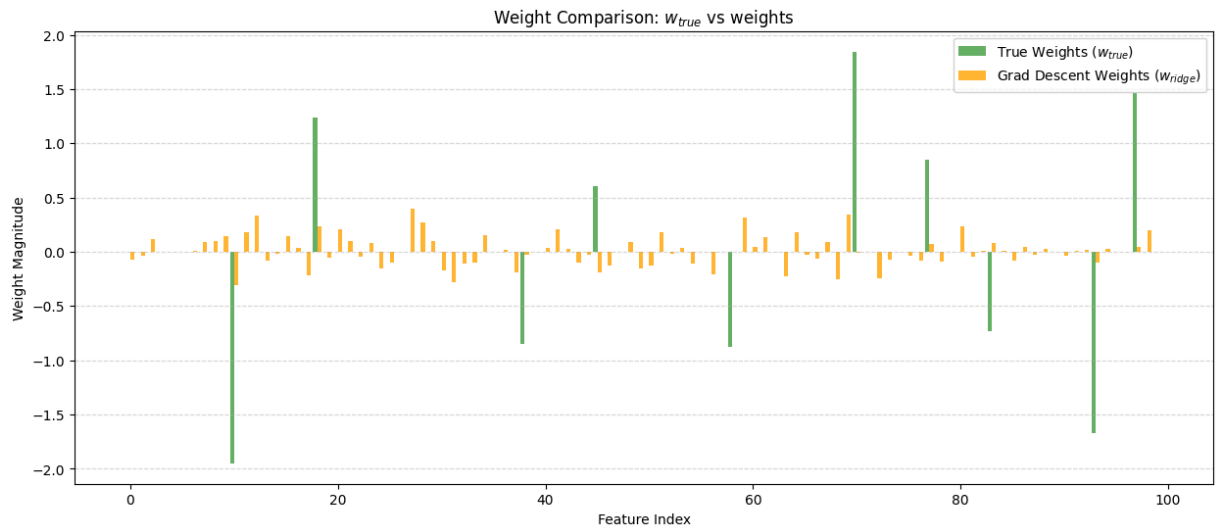
```
alpha2 = 0.001
lambda2 = [0.1, 1, 10]
for i in range(len(lambda2)):
    alpha2_post_grad_descent_weights, alpha2_post_grad_descent_losses = lasso_gradie
    final_weights = alpha2_post_grad_descent_weights[:, 0]
    print(f"weight diff: {np.linalg.norm(final_weights - w_true)}")
    # plot3(alpha2_post_grad_descent_losses, len(alpha2_post_grad_descent_losses), a
    # plt.show()

    weight_comparison(final_weights)
```

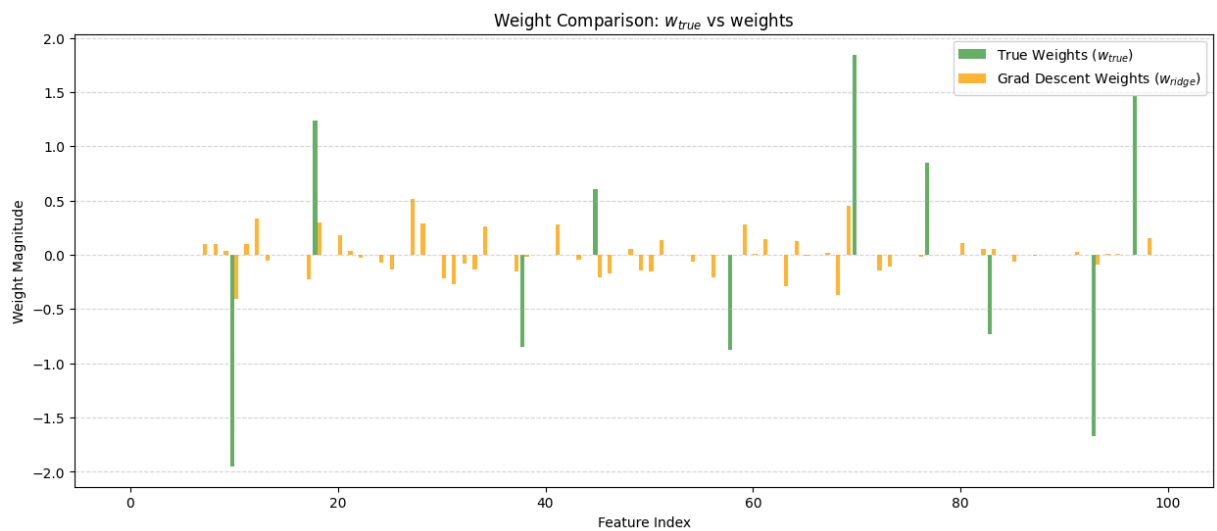


```
plt.show()
```

weight diff: 4.07294079221863



weight diff: 4.0571008615682755



weight diff: 4.125456928434533

